

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

def detect_layered_rocks_multiscale(image_path, template_crop_coords=None):
    # 1. Load the image
    img = cv2.imread('/content/seed1.jpg')
    if img is None:
        print("Error: Could not find image.")
        return

    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 2. Define the Template
    if template_crop_coords:
        y1, y2, x1, x2 = template_crop_coords
        template = gray_img[y1:y2, x1:x2]
    else:
        # Fallback crop
        h, w = gray_img.shape
        template = gray_img[int(h*0.4):int(h*0.5), int(w*0.4):int(w*0.5)]

    (tH, tW) = template.shape[:2]

    # List to store all detected boxes from all scales
    all_boxes = []

    # 3. RESEARCH PAPER CONCEPT: Multi-Scale Search
    # The paper mentions handling "varying sizes" effectively[cite: 44].
    # We loop through scales from 20% (tiny rocks) to 150% (huge rocks).
    scales = np.linspace(0.2, 1.5, 20)[::-1]

    print(f"Scanning {len(scales)} different zoom levels...")

    for scale in scales:
        # Resize the image according to the scale
        resized_w = int(gray_img.shape[1] * scale)
        resized_h = int(gray_img.shape[0] * scale)
        resized = cv2.resize(gray_img, (resized_w, resized_h))

        # Check if resized image is smaller than template (stop if so)
        if resized.shape[0] < tH or resized.shape[1] < tW:
            break

        # Calculate ratio to map coordinates back to original image
        r = gray_img.shape[1] / float(resized.shape[1])

    # 4. Apply Template Matching
    result = cv2.matchTemplate(resized, template, cv2.TM_CCOEFF_NORMED)

    # LOWER THRESHOLD slightly to catch fainter matches

```

```

threshold = 0.45
locations = np.where(result >= threshold)

# 5. Collect Boxes
for pt in zip(*locations[::-1]):
    # Scale coordinates back to original size
    startX = int(pt[0] * r)
    startY = int(pt[1] * r)
    endX = int((pt[0] + tW) * r)
    endY = int((pt[1] + tH) * r)

    all_boxes.append([startX, startY, endX - startX, endY - startY])

# 6. Apply Non-Maximum Suppression (NMS)
# This groups the thousands of boxes into clean detections
# We use groupRectangles which is OpenCV's built-in NMS
rects, weights = cv2.groupRectangles(all_boxes, groupThreshold=1, eps=0.2)

print(f"Found {len(rects)} layered formations.")

# 7. Visualization
debug_img = img.copy()
for (x, y, w, h) in rects:
    cv2.rectangle(debug_img, (x, y), (x + w, y + h), (0, 255, 0), 3)

plt.figure(figsize=(15, 10))

plt.subplot(131)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

plt.subplot(132)
plt.title("Template")
plt.imshow(template, cmap='gray')

plt.subplot(133)
plt.title("Multi-Scale Detections")
plt.imshow(cv2.cvtColor(debug_img, cv2.COLOR_BGR2RGB))
plt.show()

# Run with your specific crop coordinates
# Replace these numbers with the coordinates of your "Red Box"
my_crop = (400, 480, 430, 500)
detect_layered_rocks_multiscale('/content/seed1.jpg', template_crop_coords=my_crop)

```

